DLDCA Inlab: Is This Performative?

Week 7

November 6, 2026

Introduction

Pipelining, data hazards, forwarding and branch penalties are usually studied on paper. In this lab you watch them happen.

Tasks 1 to 3 use EduMIPS64, a simulator of the classic five-stage MIPS64 pipeline. It shows every instruction moving through the pipeline, cycle by cycle. In Task 1 you predict the cycle count and the CPI of a program by hand, then measure them. In Tasks 2 and 3 you make a loop faster by changing only the code.

Tasks 4 to 7 measure a real processor (Tasks 6 and 7 are optional). Task 7 is a practice task: debug a timing program that lies. Tasks 4 to 7 use three clocks: `std::chrono`, `clock_gettime` and the `rdtsc` instruction. They also use the Linux tool `perf`, which asks the kernel to read the processor's hardware performance counters.

This lab is ungraded (0 marks). You still submit it the usual way, so that you can practice the procedure. Nothing is tested.

Necessary tools

EduMIPS64 (Tasks 1 to 3)

You can run the MIPS programs in two ways: locally or on the web.

Option A: the web version. Open `https://web.edumips.org` in a browser. Nothing needs to be installed. Type/paste your program into the editor, or open a `.s` file with `Program` → `Open...`. Press `Load` (top right), then run the program with the toolbar buttons. The web version **cannot run the tests in the makefile**. To run the tests of Tasks 2 and 3, you need to **paste them into the editor** as described under *Test data* (see page 3) below.

Option B: the local version. The lab folder contains `edumips64`, a ready-to-run executable of EduMIPS64 (Linux). Nothing needs to be installed. Use the **makefile** to run the programs. It runs them with `./edumips64`, without a window, **on all testcases**, and prints the cycle counts (see *Building and testing*). To actually see the pipeline drawn on screen, run `./edumips64` on a machine with a display. If the executable does not run on your system (Windows, macOS, or an incompatible Linux), use Option A instead, or see *Running the jar locally* in the appendix (page 23).

The full manual is at `https://edumips64.readthedocs.io`. The relevant pages are *The instruction set*, *Source files format* and *The web user interface*. The parts you need for this lab are summarized in *An EduMIPS64 Tutorial* below.

Timing tools (Tasks 4 to 7)

You need the following tools:

```
$ gcc --version
gcc ([linux-flavour] [version]) [version]
$ g++ --version
g++ ([linux-flavour] [version]) [version]
$ nasm -v
NASM version [version]
$ perf --version
perf version [version]
```

If `perf` is missing, install it with the first command below. If `perf` then fails with a permission error, the second command lets it run without `sudo`.

```
$ sudo apt install linux-tools-common linux-tools-$(uname -r)     # Ubuntu
$ sudo sysctl -w kernel.perf_event_paranoid=1
```

If neither works, use a Software Lab machine. Log in with

`ssh <cse-username>@sl<N>-<MM>.cse.iitb.ac.in`

The slides contain more troubleshooting.

Pinning. Pin every timing run to one core (this gives you more reliable results) with `taskset -c N`, where `N` is a core number, for example 2.

The makefile pins Tasks 4 to 7 to **core 0** by default. To use core 2 instead, run `make task4 CPU=2`. If you type a `perf stat` command by hand, include `taskset -c N` yourself.

Measured values. All measurements quoted in Tasks 4 to 7 were taken on the author's laptop. Your machine will give different values. So, compare the trends, not the numbers.

An EduMIPS64 Tutorial

Note: you can read the attached `EduMIPS64-manual-en.pdf` reference file for detailed information. Here follows a summary to get started.

Source files. A program has two sections:

- `.data`: memory that the program reads and writes,
- `.text`: the instructions.

General syntax.

- An instruction placed before the `.text` line causes a parse error.
- A comment starts with `;`. A label is a name followed by a colon. The directive `.word` stores 64-bit values, and `.space n` reserves `n` bytes.
- The registers are `r0` to `r31`. Register `r0` always reads as 0.

- A memory operand is written `offset(register)`. For example, `ld r1, 8(r2)` loads from the address `r2 + 8`, and `ld r1, a(r0)` loads from the label `a`.
- The instruction `syscall 0` is **used to end the program**. It counts as an instruction. The instruction table at the end of this document lists every MIPS64 instruction used in Tasks 1 to 3.

The panels (web version).

- The **cycles diagram**, at the bottom, has one row per instruction and one column per cycle. This is **THE pipeline diagram**.
- The right-hand side has a bunch of panels which you must scroll down to see. You can drag and drop to adjust their placement btw.
- `Pipeline` shows you the full picture—what instruction is in which stage.
- `Registers` and `Memory` show the values after the last executed cycle. **Check the result of a program here**.
- `Statistics` reports the **number of cycles** and **instructions**, the CPI, and counts of **some kinds of stall**, such as RAW stalls. The cost of a "taken" branch shows up as a bubble in the cycles diagram and in the total cycle count.
- `Standard Output` shows what the program prints. We are not using this feature currently.
- The toolbar has five controls:
  - *single step* advances the pipeline by one clock cycle,
  - *multi step* advances it by several cycles,
  - *run all* runs to the end of the program,
  - *stop* interrupts a run,
  - *reset* restarts the program.

The panels (local version). The desktop version shows the same views in separate windows: `Cycles`, `Pipeline`, `Registers`, `Data` and `Statistics`.

- To open a file, use `File` → `Open`.
- To run, use `Execute` → `Single Cycle` (F7) or `Run` (F4).
- To enable forwarding, use `Configure` → `Settings` → `Main settings` → `Enable forwarding`.

Forwarding. Forwarding is **OFF by default**. To switch it on:

- **Web version:** open `General Settings` (the gear icon) and set `CPU Forwarding`.
- **Local version:** use `Configure` → `Settings` → `Main settings` (see above).

How to run tests!

Test data. The programs of Tasks 2 and 3 process arrays of N elements. Each program begins with a data section. It holds the N value at the label `n`, followed by the arrays `a`, `b` and `c`, in this order. This data section is a small test with $N = 8$.

Now, the makefile runs longer tests where it replaces the data section with generated data.

If you are on the web version. Running `make task2-tests` and `make task3-tests` produces the files `task2/tests.txt` and `task3/tests.txt`. Then to run a test, do the following:

1. In the editor, select the data section of the program (from the `.data` line up to, but not including, the `.text` line).
2. Paste a test over it, from the `tests.txt` file.
3. Run the program.
4. Compare `c` in the `Memory` panel with the expected values listed above the test.

If you are on the local version. Just run `make check2` and `make check3`.

Structure of submission/templates

Edit only the files marked `(TODO)`: `unroll2.s` and `unroll4.s` in `task2/`, `general.s` in `task3/`, and `dotprod.cpp` in `task7/`. Every other file is provided. Tasks 1, 4, 5 and 6 involve no editing.

Submission format:

```
your-roll-no/
|- task1/
    |- hazard.s
|- task2/
    |- loop.s
    |- unroll2.s (TODO)
    |- unroll4.s (TODO)
|- task3/
    |- general.s (TODO)
|- task4/
    |- matmul.c
|- task5/
    |- chrono_demo.cpp
|- task6/
    |- timing.s (optional)
|- task7/
    |- dotprod.cpp (TODO, optional)
```

Submit this folder as a tarball named `your-roll-no.tar.gz`, with the roll number in lowercase. Build it with:

```
$ ./build_archive.sh
Enter Student Roll Number: your-roll-no
Packaging assignment files...
  [+] Copied: ...
Successfully created your-roll-no.tar.gz
```

Building and testing

Run all commands from the `07_inlab` directory.

```
$ make task1                 # hazard.s, without and then with forwarding
$ make task1 args=fwd        # only with forwarding (args=nofwd: only without)
$ make task2                 # cycles for N = 8 and 4096
$ make task2-tests           # write task2/tests.txt (web version)
$ make task3                 # cycles for six sizes of N
$ make task3-tests           # write task3/tests.txt
$ make check2                # PASS/FAIL for Task 2 (check3, check)
$ make task4 args=256        # matrix multiply, N = 256, pinned to one core
$ taskset -c N perf stat ./task4.out 256   # same program, perf
$ make task5                 # std::chrono demonstration (make task5 CPU=N)
$ make task6                 # rdtsc and clock_gettime in assembly
$ make task7                 # the broken dot-product timing (make check7)
$ make clean                 # delete everything that was generated
```

`make check2` and `make check3` run the program of one task on its test sizes, which are listed in Tasks 2 and 3. `make check` runs both.

Each run prints `PASS` if the array `c` ends up holding $a[i] + b[i]$ for every element, and `FAIL` otherwise. The provided MIPS `(TODO)` files stop without writing `c`, so they fail until you complete them. Your programs must pass this check. Correctness comes before speed.

If a build keeps failing, run `make clean` to delete stale files.

Tasks

Task 1: Can You Count?

Objective: Review theory concepts, apply them to a simple problem and confirm our answers using a simulator.

Review of Pipelining, Data and Control Hazards for the EduMIPS64 Simulator

Recall: A five-stage pipeline overlaps instructions: while one instruction is in EX, the next is in ID and the one after it in IF.

Step 1: Cycles without waiting. The five stages are:

- **IF**: fetch the instruction,
- **ID**: decode it and read its registers,
- **EX**: compute,
- **MEM**: access memory,
- **WB**: write the result to the register.

One instruction needs 5 cycles. Each further instruction finishes one cycle after the previous one, so without stalls

$$\text{cycles} = 4 + n, \qquad \text{CPI} = \frac{\text{cycles}}{\text{instructions}}.$$

For example, four independent instructions take 8 cycles, so CPI $= 8/4 = 2$. CPI approaches 1 only as n grows, because the first 4 cycles are spent filling the pipeline.

Simulator note: the simulator counts cycles until the last instruction, `syscall 0`, has left WB, and it counts `syscall 0` as an instruction.

Step 2: A RAW hazard. A *read after write* (RAW) hazard occurs when an instruction (the *consumer*) reads a register written by an earlier instruction (the *producer*). The producer writes the register in WB, but the consumer needs the value in ID.

Without forwarding, the consumer waits in ID until the producer reaches WB. The read may happen in the same cycle as the write. So:

- consumer directly behind its producer: waits **2** cycles,
- one instruction in between: waits **1** cycle,
- two or more instructions in between: waits **0** cycles.

With forwarding (also called bypassing), a result skips the register file. As soon as a value is in a pipeline register, later instructions take it from there instead of waiting for WB. In this simulator:

- `dadd`, `dsub`, `daddi`: the result is passed on at the end of EX. The next instruction (ALU, load or store) uses it with **no** wait.
- `ld`: the value exists only after MEM. An instruction that uses it directly after the load waits **1** cycle.
- An `sd` that stores the loaded value does **not** wait: it needs the value to be stored only in its MEM stage, one cycle after the `ld` has produced it.

Step 3: Branches. A branch is decided in ID. The simulator has no branch predictor, so it behaves as if every branch were predicted "not taken":

- While the branch is in ID, IF fetches the next sequential instruction. When the branch is decided, that instruction is still in IF and has not changed any state.
- **Not taken**: the fetched instruction continues down the pipeline. Cost: 0 cycles.
- **Taken**: the fetched instruction is discarded and the target is fetched instead. Cost: 1 cycle.

So the instruction after a branch takes effect only when the branch is not taken.

A branch also reads its register in ID, so it waits like any other consumer. Because it needs the value already in ID, it still waits even with forwarding:

- directly after an ALU instruction: waits 1 cycle,
- directly after an `ld`: waits 2 cycles,
- one instruction between it and the `ld`: waits 1 cycle.

Table 2 at the end of this document summarizes Steps 1 to 3.

A worked example

Consider the program

```
        .text
        daddi r1, r0, 5
        dadd  r2, r1, r1     ; reads r1, written by the line above
        daddi r3, r0, 7
        syscall 0
```

Without forwarding the `dadd` waits 2 cycles, so the program takes $4 + 4 + 2 = 10$ cycles and CPI $= 10/4 = 2.5$. In the diagram each column is one cycle; `s` marks a cycle spent waiting in ID, and `.` marks an instruction held in IF behind the waiting one.

```
cycle          1  2  3  4  5  6  7  8  9 10
daddi r1,r0,5  IF ID EX ME WB
dadd  r2,r1,r1    IF ID s  s  EX ME WB
daddi r3,r0,7        IF .  .  ID EX ME WB
syscall 0                     IF ID EX ME WB
```

With forwarding there is no wait: $4 + 4 = 8$ cycles and CPI $= 8/4 = 2$.

Task. The program `hazard.s` computes $2(a + b) - a$, stores the result in memory, and then branches over an error path (if the branch were not taken, the error path would overwrite the result with -1). It contains a bunch of RAW hazards, and has one taken branch.

1. Mark every RAW pair.
2. Draw the pipeline diagram *without* forwarding, in the form used above. Count the cycles and compute the CPI.
3. Draw the diagram *with* forwarding. Count the cycles and compute the CPI.
4. Now, actually run the program in EduMIPS64 with forwarding off and then on. Compare the cycles diagram and the `Statistics` panel with the drawings, and check the counts with `make task1`.
5. Explain which stalls disappear when forwarding is enabled, which remain, and why. Explain also why the `bne` still waits when forwarding is on.

```
$ make task1
== task1/hazard.s
forwarding: false
cycles: 24
instructions: 9
CPI: 2.667

== task1/hazard.s
forwarding: true
cycles: 16
instructions: 9
CPI: 1.778
```

Task 2: Hurry Up Tomorrow

We have written loops before. There is overhead on every iteration: the branch, the counter update and the pointer updates, none of which is the work the loop actually wants to do. The idea behind **unrolling** is to repeat the body several times inside one iteration, so that this overhead is paid **once for several elements**. This task introduces unrolling on a small example and measures its effect on short and long arrays.

The program. The program `loop.s` adds two arrays element by element, $c[i] = a[i] + b[i]$, for N elements of 64 bits (8 bytes) each. The data section is described under *Test data* in *An EduMIPS64 Tutorial* (see page 3). A fixed setup block at the start of the program reads N and prepares the registers the loop uses:

- `r1`, `r2`, `r3`: pointers to the current element of a, b and c,
- `r4`: the number of elements that remain.

The loop assumes $N \geq 1$. One iteration consists of 9 instructions:

```
loop:   ld    r5, 0(r1)     ; r5 = a[i]
        ld    r6, 0(r2)     ; r6 = b[i]
        dadd  r7, r5, r6    ; r7 = a[i] + b[i]
        sd    r7, 0(r3)     ; c[i] = r7
        daddi r1, r1, 8     ; next element: three pointers ...
        daddi r2, r2, 8
        daddi r3, r3, 8
        daddi r4, r4, -1    ; ... and the counter
        bne   r4, r0, loop  ; repeat while elements remain
```

Only `dadd` performs arithmetic on the data; the other eight instructions load, store, advance pointers or control the loop.

Unrolling. *Unrolling by k* means that one iteration performs the work of k elements. The body (load, load, add, store) is written k times, and the pointers and the counter are updated once per iteration instead of once per element. For $k = 2$ the loop needs half as many iterations, and therefore half as many branches, counter updates and pointer updates. Two details require care:

- The second element of an iteration lies at offset 8 from each pointer, the third at offset 16, and so on: `0(r1)`, `8(r1)`, `16(r1)`.
- Each element needs its own registers for its two loaded values and their sum. If every copy reused `r5` to `r7`, each copy would overwrite the values of the previous one.

An unrolled loop handles k elements per iteration, so it is correct only when N is a multiple of k. Otherwise elements are left over; Task 3 treats that case.

Task. Complete the two files marked `(TODO)`:

- `unroll2.s`: the loop unrolled by 2.
- `unroll4.s`: the loop unrolled by 4.

Each file starts with the same data section and setup block as `loop.s`. **The setup block must not be changed.**

Testing. The tests use $N = 8$ and $N = 4096$, both multiples of 2 and 4.

- **Test sizes:** the data section of each file contains the test with $N = 8$; the test with $N = 4096$ is generated by the makefile.
- `make check2`: checks both programs on both sizes.
- `make task2`: prints the cycle counts of `loop.s`, `unroll2.s` and `unroll4.s` on both sizes, without and with forwarding.
- **Web version:** the tests come from `make task2-tests` (see *Test data*, page 3).

```
$ make check2
PASS -- task2/unroll2.s N=8
PASS -- task2/unroll2.s N=4096
PASS -- task2/unroll4.s N=8
PASS -- task2/unroll4.s N=4096
```

Reference results. A straightforward unrolling (each copy of the body written out in order, followed by the pointer and counter updates) needs about the following numbers of cycles. The exact counts depend on the order of the instructions.

	$N = 8$		$N = 4096$	
Program	no forwarding	forwarding	no forwarding	forwarding
`loop.s`	142	105	65550	49161
`unroll2.s`	110	77	49166	34825
`unroll4.s`	94	63	40974	27657

Questions. Important to think about.

1. For each program and each test size, compute the cycles per element (total cycles divided by N). How do the three programs compare for a given N?
2. How many instructions does one iteration of `unroll4.s` contain, and how many of them are loop overhead (branch, counter, pointers)? Compare with the 9 instructions and 5 overhead instructions of one element in `loop.s`.
3. Are the savings larger for larger N or smaller N?
4. Unrolling by 8 or 16 would save even more overhead. But is there a hidden cost we are neglecting?
5. What happens when N is not a multiple of 4? How can you deal with that? We will see this in Task 3.

Task 3: Blinding Lights

The unrolled loops of Task 2 are correct only when N is a multiple of the unroll factor, whereas real code has to work for any N. In this task you write one routine that works for every N and is as fast as possible.

Step 1: The leftover elements. A loop unrolled by 4 stops when there are 0, 1, 2 or 3 elements left over. The routine handles this by writing three loops in a row:

- **Loop 4** (unrolled by 4): runs while at least 4 elements remain.
- **Loop 2** (unrolled by 2): runs while at least 2 elements remain. Fewer than 4 are left after loop 4, so it runs at most once.
- **Loop 1** (the ordinary loop): runs while at least 1 element remains. Fewer than 2 are left after loop 2, so it runs at most once.

For example (iterations of loop 4, loop 2 and loop 1):

$$N = 9 = 2 \times 4 + 0 \times 2 + 1$$
$$N = 6 = 1 \times 4 + 1 \times 2 + 0$$
$$N = 2047 = 511 \times 4 + 1 \times 2 + 1$$

Compilers generate this shape for unrolled loops.

Step 2: Compare and jump. Each loop requires two decisions: whether to enter it at all (fewer elements may remain than one iteration handles), and whether to repeat it. Both compare the number of remaining elements `r4` with a constant.

- `slti rd, rs, imm` sets `rd` to 1 if `rs` $<$ `imm` and to 0 otherwise.
- A following `bne rd, r0, label` jumps when the condition holds, and `beq rd, r0, label` jumps when it does not.
- The branches of this lab (`beq`, `bne`) only test for equality, so every less-than test is written as `slti` (or `slt`) followed by `bne` or `beq`.

Avoiding branch stalls: the compare, and the counter update in front of it, produce the value the branch reads, so the branch rules of Task 1 (Step 3) apply. Compute the compare early and place the branch later, so that the branch does not wait. Each taken branch also costs a cycle, so keep the number of taken branches low.

Task. Complete `general.s`, marked `(TODO)`. It has the same data section and setup block as Task 2 (**the setup block must not be changed**). It must compute $c[i] = a[i] + b[i]$ with three loops in the order 4, 2, 1; the loop bodies come from `loop.s` and from Task 2. The code has two goals, in this order:

1. **Correct for every $N \geq 0$.** A schedule that is fast but wrong for one N is worthless.
2. **As fast as possible.** Unlike Task 2, schedule all three loops by the hazard rules (Task 1, Step 2) and the branch rules (Task 1, Step 3) so that RAW and other stalls are removed.

Testing. The tests use $N = 8, 6, 9, 1024, 2047$ and 4098. They cover a multiple of 4 (8, 1024) and remainders of 2 (6, 4098), 1 (9) and 3 (2047), on short and on long arrays, so that the cost of the loop overhead and of the leftover code is visible at low and at high iteration counts.

- `make check3`: checks `general.s` on these six sizes and on the edge sizes $N = 0$, 1 and 3. The provided file passes only $N = 0$.
- `make task3`: prints the cycle counts of `general.s` and, for comparison, of `loop.s` from Task 2 on the six sizes, without and with forwarding. All six sizes are at least 1, which `loop.s` requires.
- **Web version:** the tests come from `make task3-tests` (see *Test data*, page 3).
- **Trying $N = 0$ by hand:** set `n` to 0 and leave `a:`, `b:` and `c:` as bare labels with nothing after them. The program must not write outside `c`.

```
$ make check3
PASS -- task3/general.s N=8
PASS -- task3/general.s N=6
PASS -- task3/general.s N=9
PASS -- task3/general.s N=1024
PASS -- task3/general.s N=2047
PASS -- task3/general.s N=4098
PASS -- task3/general.s N=0
PASS -- task3/general.s N=1
PASS -- task3/general.s N=3
```

Reference results. A well-scheduled solution reaches about the following numbers of cycles. A difference of a few cycles is normal; a large difference indicates a stall that can still be removed.

	`loop.s`		`general.s`	
N	no forwarding	forwarding	no forwarding	forwarding
8	142	105	74	66
6	110	81	62	54
9	158	117	81	73
1024	16398	12297	5916	5908
2047	32766	24573	11799	11791
4098	65582	49185	23591	23583

Questions.

1. For each test size, compute the cycles per element of `general.s` and of `loop.s`. Which size benefits least from the unrolled code, and why?
2. Which of the six sizes takes the most cycles per element in your code? Relate this to the number of leftover elements.
3. How many stall cycles remain in each of your three loops, with and without forwarding, and which of them can be removed only by changing the structure of the code?

Task 4: After Hours

In Tasks 1 to 3 we counted cycles by hand and in a simulator. A real processor does not behave like that simulator: it can complete **several instructions per cycle**. In this task we run a real program, a matrix multiplication, under `perf`, which reads the processor's own hardware counters. You will learn to **run** `perf stat`, to **read its output**, and to compute the **CPI and IPC** of a real program.

Step 1: What `perf stat` does. Every processor core has a small set of hardware *performance counters*. Each counter can be set to count one kind of event: instructions retired (completed), core clock cycles, branch mispredictions, cache misses, and so on. The counting is done by the hardware; no software runs for each event.

The command `perf stat ./prog` works in three steps:

1. It asks the kernel to configure the counters for the program (through the `perf_event_open` system call).
2. It starts the program.
3. It reads the counters when the program exits.

What the totals include:

- **Only the program's own events.** The kernel saves and restores the counters whenever the scheduler switches to another process.
- **The whole process**, from the first instruction of the startup code to the last instruction of the exit code.
- **Time spent in the kernel** (for example, when you do a syscall, it leads to code execution in the kernel) on the program's behalf. If `perf` prints `cycles:u`, only user mode is counted.
- **Core cycles:** the reported `cycles` follow the varying clock of the core (explained in the optional Task 6).

Step 2: The program. The program `matmul.c` multiplies two $N \times N$ matrices of doubles with three nested loops. N is the first argument (default 512). The matrices are filled by a cheap formula, and only the multiplication is timed: the program measures it with `clock_gettime` and with `rdtsc` (both explained in Task 6), prints one line and exits. It is compiled without optimization (`-O0`), so the loops execute as written in the source.

Step 3: Reading the counters. The program is run under `perf`, pinned to one core (see *Pinning*); the transcript uses core 2. `make task4 args=256` runs the program pinned in the same way (core 0 by default, `CPU=2` for core 2), but without `perf`, so the `perf stat` command has to be typed by hand.

```
$ make task4.out
cc -g -O0 -c -o task4/matmul.o task4/matmul.c
g++ -no-pie -o task4.out task4/matmul.o
$ taskset -c 2 perf stat -e task-clock,cycles,instructions ./task4.out 256
N=256  tsc_ticks=765069390  tsc_MHz=3792
tsc_ns=201716775  clock_gettime_ns=201716775
checksum=4480

 Performance counter stats for './task4.out 256':

          207.27 msec task-clock
       287,175,786      cycles
       717,986,882      instructions

       0.210015394 seconds time elapsed

       0.204109000 seconds user
       0.005973000 seconds sys
```

Note. `tsc_ns` and `clock_gettime_ns` match by construction: the tick rate is measured in the same run (see Task 6), so this is only a consistency check.

Step 4: CPI and IPC. Besides the CPI = cycles / instructions, a real machine is often described by its *IPC* = instructions / cycles, the inverse of the CPI. An IPC above 1 means that the processor completed more than one instruction per cycle. This is normal for a modern core, which is:

- *superscalar*: it fetches, decodes and starts several instructions per cycle, and
- *out of order*: it executes an instruction as soon as its inputs are ready, not in program order, which keeps those issue slots filled.

The simulator issues one instruction per cycle, in order.

Task.

1. Run the program under `perf stat` for $N = 8$, 128, 256 and 512, and record `cycles`, `instructions` and `seconds time elapsed`.
2. Compute the CPI and the IPC for each run.

Questions.

1. Compare your CPI values with those of Tasks 1 to 3. To what extent do the CPI values of Tasks 1 to 3 describe a real machine?
2. The multiplication performs N^3 multiply-adds, so doubling N from 128 to 256 multiplies the work by 8. Did the time grow by about 8? Did the instruction count?
3. For $N = 8$ the multiplication is only 512 multiply-adds, yet the whole process executes about a million instructions. Where do these come from, and what does this imply for the use of `perf stat` on a very small piece of work?

Reading the results. On the author's laptop, $N = 8$ gave 1.04M instructions in 1.50M cycles, $N = 256$ gave 718M instructions in 287M cycles, and $N = 512$ took 1.65 s. Every process pays a fixed startup and exit cost for the program, the libraries and the operating system. This cost is negligible for a large multiplication and dominates a tiny one. Task 5 shows the same effect for the cost of reading a clock.

Task 5: Too Late

To time a piece of code, we read a clock before it, read the clock again after it, and subtract. This works only if the code takes much longer than reading the clock itself. In this task we time **a single** `add` with `std::chrono` and see that the result is **mostly the cost of the clock**. You will learn how to time **very short code** correctly: **repeat it many times and divide**.

Step 1: The clock. The class `std::chrono::steady_clock` is the C++ clock intended for measuring intervals: it never goes backwards and does not jump when the system time is changed. The call `steady_clock::now()` returns a time point, the difference of two time points is a duration, and `std::chrono::duration<double, std::nano>` converts a duration to nanoseconds.

Step 2: The experiment. The program `chrono_demo.cpp` measures the same operation, one `add`, in two ways:

1. **Once**: `now()`, one add, `now()`, subtract.
2. **Repeated**: `now()`, the add 1,000,000 times, `now()`, subtract, divide by 1,000,000.

Reading the clock itself takes time (about 20 ns on the author's laptop; Task 6 compares ways of reading a clock). In the first measurement this cost lies *inside* the interval and is far larger than the add. In the second it is paid once and shared among a million adds, so it contributes less than 0.001 ns to each. The result is kept in a `volatile` variable so that the compiler cannot delete the add; the appendix entry *Timing trickeries* (page 23) explains why. The program reads the clock once before the first measurement, because the very first call is slower than the following ones.

Task.

1. Run `make task5` three times and record the "single add" and "repeated add" values of each run.
2. What does the single-add value measure? By what factor is it larger than the repeated value?
3. Give one reason to prefer the repeated measurement, and one reason why it is still not exactly the cost of an add (consider the loop in the source).

```
$ make task5
taskset -c 0 ./task5.out
single add  : 20.00 ns (timer overhead dominates)
repeated add: 0.43 ns per iteration (over 1000000 reps)
sink=1000001

Return Code: 0
```

Task 6: You Wanted More Assembly? (optional)

This task is optional. It is a deep dive, in assembly, into what Task 4 already does: reading the TSC, converting ticks to nanoseconds, and comparing the result with `clock_gettime`. Nothing in Tasks 1 to 5 depends on it.

In Tasks 4 and 5 we used clocks without looking inside them. Here we read time in assembly, in two ways: with the **instruction** `rdtsc` and with the **system call** `clock_gettime`. The program `timing.s` times one loop with both. You will learn what `rdtsc` counts, how to **convert its ticks to nanoseconds**, and why an `lfence` goes in front of it.

Step 1: `rdtsc`. The processor has a 64-bit counter, the time stamp counter (TSC). The instruction `rdtsc` reads it. The low 32 bits go to `eax` and the high 32 bits to `edx`. The program joins them with `shl rdx, 32` and `or rax, rdx`. The difference of two readings is a number of ticks.

TSC ticks are not CPU cycles. The clock of a core speeds up and slows down. The TSC ignores this and ticks at a fixed rate. So the TSC is a stopwatch: it measures time. The `cycles` that `perf` reports are core cycles, a different count. We turn ticks into time in Step 3.

A machine with such a fixed-rate TSC has the flags `constant_tsc` and `nonstop_tsc`. This command should print both (the `Flags` line of `lscpu` shows them too):

```
$ grep -o -w -e constant_tsc -e nonstop_tsc /proc/cpuinfo | sort -u
```

Put an `lfence` in front. The processor runs instructions out of order. A bare `rdtsc` can run before the code we time has finished. `lfence` waits until all earlier instructions have completed, so `timing.s` puts one in front of each `rdtsc`. `matmul.c` does the same with `_mm_lfence()`. The instruction `rdtscp` does a similar job on its own. `lfence` followed by `rdtsc` is the simplest correct form. The slides show the complete sequence and when `rdtscp` is better. The appendix entry *Timing trickeries* (page 23) explains why.

Step 2: `clock_gettime` as a system call. `clock_gettime` is Linux system call number 228. `timing.s` calls it directly with the `syscall` instruction, which always enters the kernel. The C library function `clock_gettime()`, which `std::chrono` uses, normally does not. The kernel maps a small piece of code, the *vDSO*, into every process, and the library calls that code to read the clock. On the author's laptop the vDSO call took about 18 ns and the raw system call about 150 ns. (On a virtual machine the library may fall back to the real system call.) The appendix entry *Timing trickeries* (page 23) explains what a system call does.

`timing.s` makes the call with these registers:

- `rax`: 228, the system call number,
- `rdi`: the clock identifier (1 is `CLOCK_MONOTONIC`),
- `rsi`: the address of a `timespec`; the kernel stores the seconds at offset 0 and the nanoseconds at offset 8.

The `syscall` instruction overwrites `rcx` and `r11`, so the program keeps nothing in them across the call. The elapsed time between two readings is

$$\text{ns} = (\text{sec}_1 - \text{sec}_0) \times 10^9 + (\text{nsec}_1 - \text{nsec}_0).$$

The tables at the end of this document list the registers.

Step 3: Ticks to nanoseconds. `timing.s` wraps one loop of 100 million iterations in both clocks. The TSC ticks at a fixed rate, so ticks per nanosecond is a constant of the machine. We write it in MHz (ticks per microsecond):

$$\text{MHz} = \text{ticks} \times 1000 \,/\, \text{ns}, \qquad \text{ns} = \text{ticks} \times 1000 \,/\, \text{MHz}.$$

The program prints four lines:

- `rdtsc ticks`: the ticks of the loop,
- `tsc MHz`: the rate, computed from the ticks and the nanoseconds of `clock_gettime`,
- `tsc_ns`: the ticks converted back to nanoseconds with that rate, as ticks / MHz × 1000 in double precision, so no precision is lost,
- `clock_gettime_ns`: the nanoseconds from the system call.

The last two lines match because the MHz comes from this same run, so the match is only a consistency check of the conversion. The useful part is that `tsc MHz` stays the same from run to run while the ticks and nanoseconds change. Once we know it, we can convert any tick count on this machine to time.

```
$ make task6
taskset -c 0 ./task6.out
rdtsc ticks      : 290545986
tsc MHz          : 3792   (ticks per microsecond)
tsc_ns           : 76605983
clock_gettime_ns : 76605983

Return Code: 0
```

Reading the rate directly. Instead of computing the rate from a run, you can read the value that the kernel measured at boot. It is in the kernel log:

```
$ journalctl -k -b | grep -i "tsc:"
tsc: Detected 3793.120 MHz processor
tsc: Refined TSC clocksource calibration: 3792.837 MHz
```

The refined value is the TSC rate. If `journalctl` is not allowed, try `sudo dmesg | grep -i tsc`. Do not use the `cpu MHz` in `/proc/cpuinfo`: it is the current core clock, which changes with load.

Task.

1. Run `make task6` three times and record `rdtsc ticks`, `clock_gettime_ns` and `tsc MHz`.
2. Run `make task4 args=8` several times and record the printed `tsc_MHz`.

Questions.

1. Does `tsc MHz` stay the same across your runs of `make task6` while `rdtsc ticks` and `clock_gettime_ns` change?
2. Take `tsc_ticks` from a run of Task 4 and convert it to nanoseconds with the measured `tsc MHz`. How close is the result to `clock_gettime_ns` of that run?

3. In Task 4, `perf` reported 287M `cycles` for a run whose multiplication took 765M TSC ticks ($N = 256$). Both counts grow as time passes. Compute the average clock rate of the core (cycles divided by the `task-clock` time of 0.207 s) and compare it with `tsc MHz`. Why do the two counts differ? (Hint: the clock of the *core* speeds up and slows down to save power; the TSC does not.)

4. For $N = 8$ the timed region lasts only 1 to 3 microseconds. The printed `tsc_MHz` is somewhat lower than for $N = 256$ and varies between runs (3614 to 3662 on the author's laptop, against 3792). Why does such a short region give a lower estimate?

Which clock for what. The four tools of Tasks 4 to 6 answer different questions.

Tool	Measures	Use it for
`perf stat`	hardware counters for the whole process: core cycles, instructions retired, time	how many instructions and cycles a program used, and its CPI
`std::chrono`	elapsed time between two points in the code, in ns	timing a region of C++ code
`clock_gettime`	nanoseconds; about 18 ns per call through the vDSO, about 150 ns through the raw system call (author's laptop)	the same purpose as `std::chrono`, from C or assembly
`rdtsc`	ticks at a fixed rate; converted with the MHz factor	shorter regions than the other clocks allow, because a read costs only tens of cycles; in assembly or C

The three clocks (every tool except `perf stat`) need a region that is long enough, or repeated often enough, that the cost of reading the clock is small next to it (Task 5).

Extra: how the kernel is called. The appendix entry *Timing trickeries* (page 23) explains what happens when a system call switches from the program into the kernel and back.

Task 7: Can't Feel My Face (optional practice)

This task is optional practice. It applies what Tasks 4 to 6 teach. Nothing else depends on it.

The program `dotprod.cpp` times a dot product of two vectors with 100000 elements each, and it prints a time that cannot be right. In this task we find the **three mistakes** in how it is timed and fix them.

Run `make task7`. It compiles the program with `-O2`, like real code:

```
$ make task7
taskset -c 0 ./task7.out
dot: 81.0 ns per call

Return Code: 0
```

100000 multiply-adds in 81 ns are not believable. The command `make check7` prints `FAIL` for a time below 20000 ns and `PASS` above.

Hints. Each hint points at one mistake. Try one at a time and watch the time after each change.

1. One measurement is short and noisy. How did Task 5 get a steady number?
2. If the program never uses a result, does the compiler have to compute it? Does `(void)result;` count as using it? Try it.
3. Some compilers notice that repeated calls with the same inputs give the same answer. How can we make every call count? See *Timing trickeries* (page 23).

Task. Fix `dotprod.cpp`, marked `(TODO)`. Do not change the function `dot` or the contents of the vectors. Stop when `make check7` prints `PASS`. The times below are from the author's laptop.

```
$ make check7
FAIL -- task7: 71.0 ns is too fast for 100000 multiply-adds
$ make check7
PASS -- task7: 68327.5 ns per dot product is plausible
```

Question. A plausible time can still be wrong. Compile the fixed program without optimization and compare:

```
$ g++ -O0 -std=c++17 task7/dotprod.cpp -o task7.out
$ ./task7.out
```

How much faster is the `-O2` version, and what did the broken `-O2` version claim?

Table 1: MIPS64 instructions and directives used in Tasks 1 to 3 (EduMIPS64)

Instruction	Operands	Effect
`ld`	`rt, off(rs)`	`rt` ← 64-bit value at address `rs + off`
`sd`	`rt, off(rs)`	64-bit value at address `rs + off` ← `rt`
`dadd`	`rd, rs, rt`	`rd` ← `rs` + `rt`
`dsub`	`rd, rs, rt`	`rd` ← `rs` − `rt`
`daddi`	`rt, rs, imm`	`rt` ← `rs` + `imm`
`dsll`	`rd, rt, sa`	`rd` ← `rt` shifted left by `sa` bits
`slt`	`rd, rs, rt`	`rd` ← 1 if `rs` < `rt` (signed), else 0
`slti`	`rt, rs, imm`	`rt` ← 1 if `rs` < `imm` (signed), else 0
`beq / bne`	`rs, rt, label`	jump to `label` if `rs` = / ≠ `rt`
`j`	`label`	jump to `label`
`nop`		do nothing (still takes a pipeline slot)
`syscall 0`		end the program
`.data / .text`		start the data / instruction section
`.word`	`v1, v2, ...`	store 64-bit values
`.space`	`n`	reserve `n` bytes (zeroed)

A label is a name followed by a colon. A comment starts with `;`. Registers are `r0`–`r31`; `r0` is always 0. The instruction after a branch runs only if the branch is not taken.

Table 2: Pipeline rules of the EduMIPS64 model (five stages: IF, ID, EX, MEM, WB)

Situation	Forwarding off	Forwarding on
Cycle count	4 + instructions executed + stall cycles + branch bubbles	
Consumer right after its producer (`dadd` then `dadd`)	2 stall cycles	0
One instruction in between	1 stall cycle	0
Two or more in between	0	0
`ld`, then the next instruction uses the loaded register	2 stall cycles	1
Register read by `sd` (the value it stores) or by a branch	same as any consumer: the read happens in ID	same, except: an `sd` storing a just-loaded value costs 0; a branch right after an ALU instruction (`dadd`, `dsub`, `daddi`) that computes its register costs 1; a branch right after an `ld` costs 2, and with one instruction in between costs 1
Branch taken	1 bubble: the instruction fetched behind it is thrown away	
Branch not taken	0	

The branch is decided in ID. There is no branch predictor: fetch continues with the next sequential instruction, which is discarded if the branch is taken. "Producer" means the instruction that writes the register; "in between" counts instructions between the producer and the consumer. Every program here ends with `syscall 0`, and the count runs until it leaves WB. `dmul` and `ddiv` take several cycles in EX and are not used in this lab.

Table 3: Some basic x86-64 assembly instructions (Task 6)

Instruction	Operands	Effect
`mov`	`reg, reg/*mem/imm`	`reg` ← `reg/*mem/imm`
`mov`	`*mem, reg`	`*mem` ← `reg`
`movzx / movsx`	`reg, reg/*mem`	`reg` ← `reg/*mem`, zero-/sign-extended
`lea`	`reg, *mem`	`reg` ← address of `*mem`
`add`	`reg, reg/*mem/imm`	`reg` ← `reg` + `reg/*mem/imm`
`add`	`*mem, reg/imm`	`*mem` ← `*mem` + `reg/imm`
`sub`	`reg, reg/*mem/imm`	`reg` ← `reg` − `reg/*mem/imm`
`sub`	`*mem, reg/imm`	`*mem` ← `*mem` − `reg/imm`
`inc`	`reg/*mem`	`reg/*mem` ← `reg/*mem` + 1
`dec`	`reg/*mem`	`reg/*mem` ← `reg/*mem` − 1
`neg`	`reg/*mem`	`reg/*mem` ← −`reg/*mem` (Two's complement)
`not`	`reg/*mem`	`reg/*mem` ← ¬`reg/*mem` (One's complement/bitwise)
`shl / shr`	`reg/*mem, imm/cl`	Logical shift left / right by `imm` or `cl` bits
`sar`	`reg/*mem, imm/cl`	Arithmetic shift right (preserves sign bit)
`imul`	`reg, reg/*mem/imm`	`reg` ← `reg` × `reg/*mem/imm`
`imul`	`*mem, reg/imm`	`*mem` ← `*mem` × `reg/imm`
`div`	`reg`	`rax` ← `(rdx:rax)`/`reg` `rdx` ← `(rdx:rax)`%`reg`
`and`	`reg, reg/*mem/imm`	`reg` ← `reg` ∧ `reg/*mem/imm`
`and`	`*mem, reg/imm`	`*mem` ← `*mem` ∧ `reg/imm`
`or`	`reg, reg/*mem/imm`	`reg` ← `reg` ∨ `reg/*mem/imm`
`or`	`*mem, reg/imm`	`*mem` ← `*mem` ∨ `reg/imm`
`xor`	`reg, reg/*mem/imm`	`reg` ← `reg` ⊕ `reg/*mem/imm`
`xor`	`*mem, reg/imm`	`*mem` ← `*mem` ⊕ `reg/imm`
`cmp`	`reg/*mem, reg/imm`	Sets flags based on subtraction: `reg/*mem` - `reg/imm`. ZF = 1 if equal, SF = 1 if result negative, CF = 1 if borrow (unsigned), OF = 1 if signed overflow.
`test`	`reg/*mem, reg/imm`	Sets flags based on bitwise AND: `reg/*mem` ∧ `reg/imm`. ZF = 1 if result is zero, SF = 1 if high bit set, CF and OF are cleared. No result is stored.
`jmp`	`label`	Unconditional jump to label
`je / jne`	`label`	Jump if (not) equal / zero
`ja / jae`	`label`	Jump if above / above-or-equal (unsigned)
`jb / jbe`	`label`	Jump if below / below-or-equal (unsigned)
`jg / jge`	`label`	Jump if greater / greater-or-equal (signed)
`jl / jle`	`label`	Jump if less / less-or-equal (signed)
`cmov`*cc*	`reg, reg/*mem`	`reg` ← `reg/*mem` if *cc* holds, else unchanged.
`syscall`		Performs a syscall with syscall number stored in `rax` Arguments passed in other registers starting from `rdi`
`push`	`reg`	Pushes the value in `reg` onto the stack
`pop`	`reg`	Pops the value on top of the stack onto `reg`
`call`	`label`	Calls the function called `label`
`ret`		Pops the return address from stack and jumps to it

Table 4: Timing tools used in Tasks 4–7

Tool	Description
`taskset -c N perf stat ./prog`	Hardware counters for the whole process. Add `-e task-clock,cycles,instructions` to choose the events.
`std::chrono::steady_clock::now()`	C++: a monotonic time point; subtract two and convert the difference to nanoseconds with `std::chrono::duration<double, std::nano>`.
`clock_gettime(CLOCK_MONOTONIC, &ts)`	C: monotonic time (it never jumps) in `ts.tv_sec` and `ts.tv_nsec`.
`__rdtsc()`	C (`<x86intrin.h>`): current time stamp counter, in ticks. Put `_mm_lfence()` before it, as `matmul.c` does (see page 23).

Table 5: Assembly: `rdtsc` and the `clock_gettime` syscall (Task 6)

Syscall	`rax`	Arguments	Returns (`rax`)
`clock_gettime`	228	`rdi` = clock id (`CLOCK_MONOTONIC` is 1), `rsi` = pointer to a `timespec`	0 on success

`struct timespec` is two 8-byte fields: `tv_sec` at offset 0, `tv_nsec` at offset 8. A syscall is invoked with the `syscall` instruction: number in `rax`, arguments in `rdi, rsi, rdx, r10, r8, r9`, result in `rax`. The instruction overwrites `rcx` and `r11`.

`rdtsc` takes no operands. It puts the low 32 bits of the counter in `eax` and the high 32 bits in `edx`; combine them with `shl rdx, 32` and `or rax, rdx`. Put `lfence` in front of it (or use `rdtscp`) so that all earlier instructions have finished before the counter is read (see page 23).

Ticks to nanoseconds. On one machine the counter ticks at a fixed rate. If it ticks M million times per second (printed as `tsc_MHz` by `task4.out` and as `tsc MHz` by `task6.out`), then $\text{ns} = \text{ticks} \times 1000/M$.

Appendix: notes

Timing trickeries

Three things that show up when we time code:

- **System calls.** A system call switches from the program into the kernel and back, so it costs much more than a function call (about 150 ns for a raw `clock_gettime` against about 18 ns for the C library version, on the author's laptop).
- `volatile`. It forces the compiler to perform every store to the variable, so the work behind the stored value cannot be deleted or merged. It is a blunt tool.
- `lfence`. The processor may run instructions out of order, so a bare `rdtsc` can run before the timed code has finished. An `lfence` in front of it makes it wait; `rdtscp` has a similar effect.

This is not needed for the tasks. For interested readers, an optional video: `https://www.youtube.com/watch?v=EU_nQh8wg5A`

Running the jar locally

The executable `edumips64` runs on x86-64 Linux only. On Windows, on macOS, or on a Linux system where the executable does not run, use the jar instead.

1. Install Java 17 or newer for your system, for example from `https://adoptium.net`. The steps depend on your system; ask a TA if you need help. Check the installation with `java -version`.
2. Download `edumips64-1.4.0.jar` into the lab folder (it may already be there): `https://github.com/EduMIPS64/edumips64/releases/download/v1.4.0/edumips64-1.4.0.jar`
3. Run the jar. This opens the simulator window; load a program with `File` → `Open`.

```
$ java -jar edumips64-1.4.0.jar
```

On Linux, the makefile uses the jar automatically when `./edumips64` does not work and `java` is installed. If none of this works, use the web version (Option A).